

ESERCIZIO SCRITTO 1

(?) = NON SONO SICURO SIA CORRETTO

PRIMA PARTE

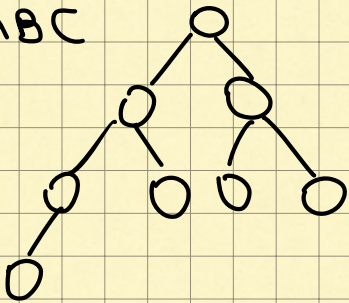
1. [3 punti] Sia $\mathcal{A}$ un algoritmo per un problema computazionale Π . Si supponga che per ogni n esista un'istanza di taglia n che richiede $\geq n^2$ operazioni. Cos'altro si deve provare per dimostrare che la complessità al caso peggio di $\mathcal{A}$ è $\Theta(n^2)$?
2. [4 punti] Dimostrare che un albero binario completo con n nodi ha altezza $h = \lfloor \log_2 n \rfloor$.
3. [5 punti] Nell'inserimento di una nuova entry in un (2,4)-Tree, un nodo v può andare in *overflow*.
 - (a) Cosa si intende per overflow di un nodo?
 - (b) Descrivere brevemente la procedura $\text{Split}(v)$ usata per sanare la condizione di overflow analizzandone la complessità.
4. [4 punti] Si consideri un grafo $G = (V, E)$.
 - (a) Dare la definizione di *spanning forest* di G .
 - (b) Si supponga che G abbia 3 componenti connesse, G_1, G_2 e G_3 . Detti n_i il numero di vertici e m_i il numero di archi di G_i , siano: $n_1 = 5, m_1 = 10$; $n_2 = 6, m_2 = 15$; $n_3 = 8, m_3 = 8$. Qual è il numero massimo di archi in una spanning forest di G ? Motivare la risposta.

ES 1

→ si deve provare che $\exists$ un'istanza di taglia n che richiede $= n^2$ operazioni $\Rightarrow \Omega(n^2)$
 $\Rightarrow \Theta(n^2)$ (perché già dimostrato che $O(n^2)$)

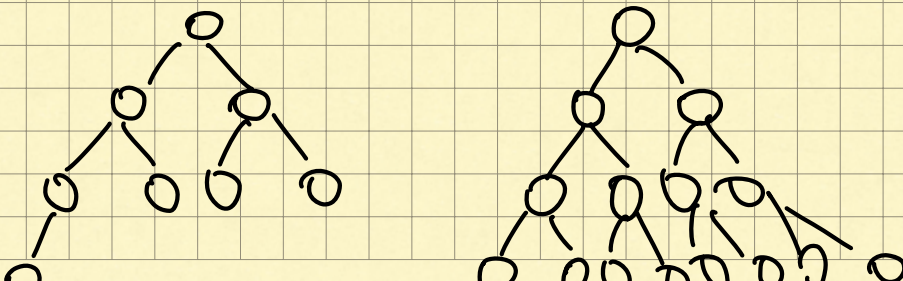
ES 2

ABC



$$h = \lfloor \log_2(n) \rfloor$$

→ LB e UB al # nodi:



$$1 + \sum_{i=0}^{R-1} 2^i \leq n \leq \sum_{i=0}^R 2^i$$

$$\sum_{i=0}^n a^i = \frac{a^{n+1} - 1}{a - 1}$$

$$1 + 2^R - 1 \leq n \leq 2^{R+1} - 1$$

$$2^R \leq n < 2^{R+1} \Rightarrow n \geq 2^R \Leftrightarrow R \leq \log_2 n$$

$$n < 2^{R+1} \Leftrightarrow R+1 < \log_2(n) + 1$$

$$\Rightarrow R \in \lfloor \log_2(n) \rfloor$$

ES3

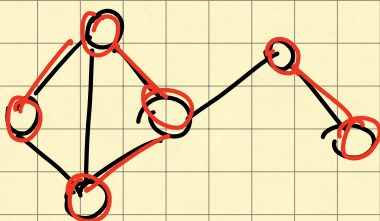
(a) $\rightarrow$ per overflow di un nodo si intende che il d-nodo contiene $> d-1$ ENTRY

(b) $\rightarrow$ NON RO SUDATO split()

?

ES4

$G = (V, E)$



(a) $\rightarrow$ una SPANNING FOREST di G è un SOTTOGRAFO di COPERTURA G' di G (con $V' = V$) NON NECESSARIAMENTE CONNESSO e SENZA CICLI

(b) $\rightarrow G = G_1 + G_2 + G_3$

$\rightarrow$ Sapendo che G_1, G_2, G_3 sono le COMPONENTI CONNESSE di $G \Rightarrow V = V_1 \cup V_2 \cup V_3, E = E_1 \cup E_2 \cup E_3$
e che: per la PROPRIETÀ 5 dei GRAFI:

"se $G = \text{FORESTA} \Rightarrow m \geq n - 1$ "

?

- $n = n_1 + n_2 + n_3 = 5 + 6 + 8 = 19$
- $m \geq n - 1 \Rightarrow m_{\text{MAX}} = 19 - 1 = 18$

SECONDA PARTE

1. [5 punti] Sia A una matrice $n \times m$ con valori interi, con la proprietà che i valori in ogni colonna sono ordinati in senso decrescente dalla riga 0 alla riga $n - 1$. Il seguente algoritmo determina il massimo numero di valori > 0 in una colonna.

```
i ← 0; j ← 0;
while (i < n) AND (j < m) do{
  if (A[i,j] ≤ 0) then j ← j + 1;
  else i ← i + 1;
}
return i;
```

Trovare un opportuno invariante per il ciclo, che serva per provare la correttezza dell'algoritmo (la prova di correttezza non è richiesta).

2. [6 punti] Sia T un albero binario di ricerca le cui entry rappresentano studenti di un'università. Ogni studente è associato a una entry (k, x) , dove k è la matricola, e x indica se lo studente è straniero ($x = 1$) o italiano ($x = 0$). Per ogni nodo $v \in T$ esiste un intero $v.\text{numStr}$ che riporta il numero di studenti stranieri in T_v (sottoalbero con radice v). Si vuole progettare un algoritmo ricorsivo MinMatStraniero per determinare la più piccola matricola di uno studente straniero in T . Se non ci sono studenti stranieri in T_v l'algoritmo restituisce null.

(a) Descrivere tramite pseudocodice la generica invocazione di MinMatStraniero, specificandone con attenzione l'input e l'output.

(b) Analizzare la complessità di MinMatStraniero

3. [5 punti] Sia S una sequenza di n chiavi distinte $\in [0, 10n]$. Progettare un algoritmo di complessità lineare che restituisca l'indice i della mediana, cioè tale che esistono esattamente $\lfloor n/2 \rfloor$ chiavi più piccole di $S[i]$ in S .

ES 1

⇒ ESEMPIO: $n = 5, m = 4$

	0	1	2	3	4
0	1	4			
1	2	3			

j ← 0
i ← 1

	2	3
2	-3	-1
3	-4	-2

→ MAX # VALORI > 0 in una colonna

?

Inv(i): alla fine dell' iterazione i del ciclo,
 $i \leftarrow$ colonna con MASSIMO # di VALORI > 0 tra
 le prime $[0 \div i]$ COLONNE di $A[n, m]$,
 $j \leftarrow |VALORI > 0| - 1$ della colonna con MASSIMO #
 di VALORI > 0 tra le prime $[0 \div i]$ COLONNE
 di $A[n, m]$

→ ES 2

2. [6 punti] Sia T un albero binario di ricerca le cui entry rappresentano studenti di un'università. Ogni studente è associato a una entry (k, x) , dove k è la matricola, e x indica se lo studente è straniero ($x = 1$) o italiano ($x = 0$). Per ogni nodo $v \in T$ esiste un intero $v.\text{numStr}$ che riporta il numero di studenti stranieri in T_v (sottoalbero con radice v). Si vuole progettare un algoritmo ricorsivo MinMatStraniero per determinare la più piccola matricola di uno studente straniero in T . Se non ci sono studenti stranieri in T_v l'algoritmo restituisce null.

(a) Descrivere tramite pseudocodice la generica invocazione di MinMatStraniero, specificandone con attenzione l'input e l'output.

(b) Analizzare la complessità di MinMatStraniero

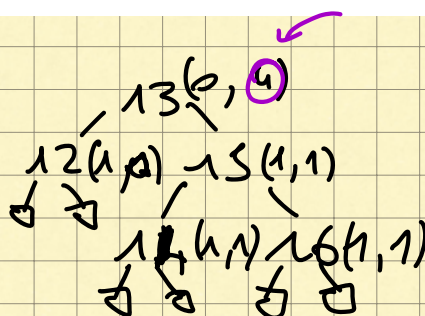
→ SPECIFICHE:

• $T \leftrightarrow \text{ABR} \leftrightarrow \text{Search Tree}$
 ABR

→ TreeSearch $\in \Theta(R) \in \Theta(n)$

• entry (k, x) $k = \text{MATRICOLA}$, $x = \begin{cases} 1 \leftrightarrow \text{STR} \\ 0 \leftrightarrow \text{ITA} \end{cases}$

• $v.\text{numStr} = \# \text{STUDENTI STR in } T_v$



- numFlatStr $\rightarrow$ + PICCOLA RAMICOLA di uno SWOONW STANISLAW, null altrimenti

$\Rightarrow$ IDCA:

- TreeSearch $\Rightarrow$ chiama sottoalbero sx di v $<$ chiave di v , chiama sottoalbero dx di v $>$ chiave di v
- visita in order (perché ABP)
- se $v.\text{numStr} = 0$
se
- altrimenti

```
if (T.left(v).numStr > 0) then {
    return numFlatStream(T, T.left(v));
}
```

```
if (v.getValue() = 1) then return v.getKey();
```

```
if (T.right(v).numStr > 0) then {
    return numFlatStream(T, T.right(v);
}
```

```
return null
```

$\Rightarrow$ COMPLESSITA'

$\Rightarrow$ VISITA in ORDER $\Rightarrow \Theta(n)$

ES 3

3. [5 punti] Sia S una sequenza di n chiavi distinte $\in [0, 10n]$. Progettare un algoritmo di complessità lineare che restituisca l'indice i della mediana, cioè tale che esistono esattamente $\lfloor n/2 \rfloor$ chiavi più piccole di $S[i]$ in S .

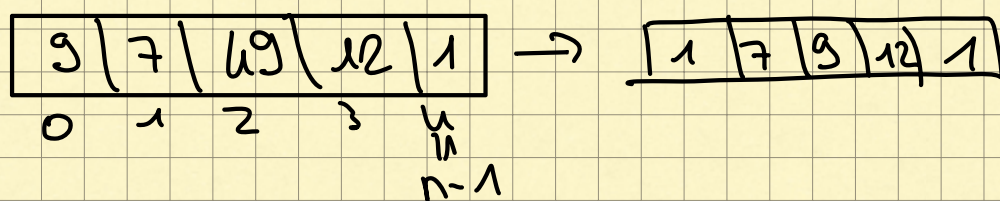
→ SPECIFICHE

→ $S = S[0], \dots, S[10n]$, n CHIAVI

→ COMPLESSITÀ $\Theta(n)$

→ OUTPUT: $i = \text{MEDIANA} = \exists \lfloor n/2 \rfloor$ chiavi + piccole di $S[i]$ in S

→ ESEMPIO



$$n = 5, \lfloor 5/2 \rfloor = 2 \Rightarrow$$

→ IDEA:

- chiavi in un RANGE $[0, N]$ suggerisce

BUCKET SORT o RADIX SORT

$$\Theta(\underbrace{n}_M + N) \in \Theta(n) \text{ perché } N = 10n$$

- poi restituire $\lfloor n/2 \rfloor$

- A: mediana(S)

- I: $S = \text{sequenza } S[0], \dots, S[n-1]$ di $n \geq 0$ chiavi nel RANGE $[0, N-1]$ dove $N-1 = 10n \Rightarrow N = 10n+1$

• c : indice della MEDIANA

// BUCKET SORT

// 1 SCAN SIONE f_i 's $\rightarrow$ BUCKET:

crea un array B di N liste: $B[0], \dots, B[N-1]$

for $i \leftarrow 1$ to $n-1$ do {

$k \leftarrow S[i];$

$B[k].\text{enqueue}(S[i]);$

}

// 2 SCAN SIONE del BUCKET lists:

$i \leftarrow 0$

for $k \leftarrow 0$ to $N-1$ do {

 while(! $B[k].\text{isEmpty}()$) do {

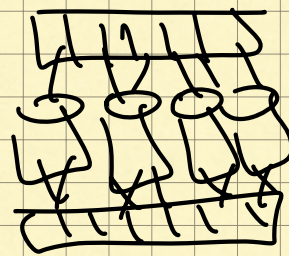
$S[i++] \leftarrow B[k].\text{dequeue}();$

 }

return $L[n/2]$

$\rightarrow$ COMPLESSITA' $\in \Theta(n+N)$ (per BUCKET SORT), ma

$N = 10n-1 \Rightarrow \in \Theta(n)$



ESERCIZIO SCRITTO 2

PRIMA PARTE

1. [4 punti] Sia A un algoritmo ricorsivo. Descrivere brevemente come, usando l'albero della ricorsione, si può dimostrare che la sua complessità al caso pessimo $t_A(n)$ è $O(f(n))$, per una data funzione $f(n)$.
2. [4 punti] Sia T un albero binario proprio non vuoto di altezza h con n nodi di cui m sono foglie. Sapendo che $m \geq h + 1$:
 - (a) Disegnare un albero T in cui il bound vale con uguaglianza ($h = 4, m = 5$).
 - (b) Dimostrare che $n \geq 2h + 1$, usando, senza dimostrarla, la relazione nota tra numero di nodi interni e foglie.
3. [4 punti] Con riferimento alla Mappa:
 - (a) Dare la specifica precisa del metodo put.
 - (b) Supponendo di implementare la Mappa tramite lista position-based, indicare la complessità di put giustificando la risposta.
4. [4 punti] Il seguente algoritmo ordina una sequenza S di n interi.

$S_1 \leftarrow$ interi pari di S ; $S_2 \leftarrow$ interi dispari di S ;
Ordina S_1 con MergeSort;
Ordina S_2 con QuickSort deterministico;
Merge($S_1, S_2; S$);

Osservando che la separazione di pari e dispari può essere fatta in tempo $\Theta(n)$, determinare la complessità al caso pessimo dell'algoritmo esprimendola con $\Theta(\cdot)$.

ES 1 :

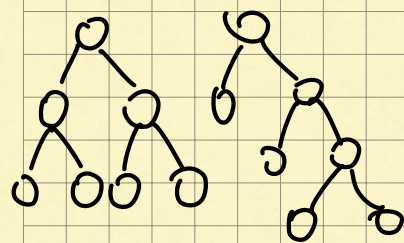
$\rightarrow$ si dimostra che e' A della R. ha :
 $\leq c_1 \cdot g(n)$ NODI di costo $c_2 \cdot R(n)$
dove $g(n) \cdot R(n) = f(n)$

ES 2 :

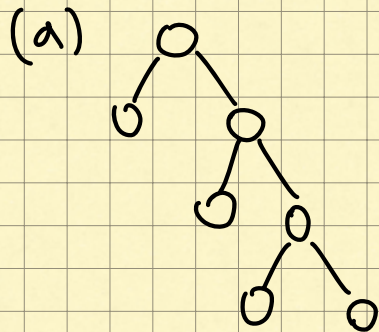
$\rightarrow T$ ABP $\Rightarrow$ ① $m = n - m + 1$

② $h+1 \leq m \leq 2^h$

③ $\log(n+1) - 1 \leq h \leq \frac{n(n-1)}{2}$



- altezza, n nodi, m foglie
- $m \geq R+1 \Rightarrow m = R+1 \Leftrightarrow T$ SBILANCIATO:



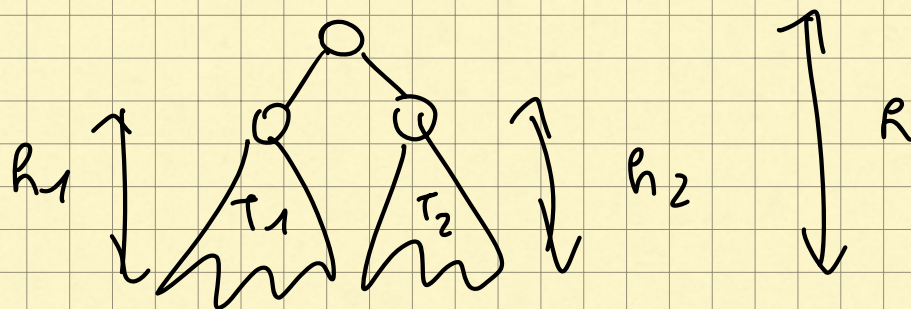
(b) DIMOSTRARE $n \geq 2R+1$ usando @ $m = n - m + 1$

$\Rightarrow$ per INDUZIONE su R ($R_0 = 0, R = 0$)

- CB: $R=0 \Rightarrow$  $T \equiv r$ $n \geq 2R+1$
 $1 \geq 1 \quad \square$

- PI: $\forall R': R_0 \leq R' \leq R$ vale che $n' \geq 2R'+1$
 $\Rightarrow$ vero per $R+1 \Leftrightarrow n \geq 2(R+1)+1$

$\Rightarrow$ considero ABPT di altezza $R+1$



$\Rightarrow$ per def di altezza:

$$\text{height}(T) = 1 + \max\{h_1, h_2\}$$

$$R+1 = 1 + \max\{h_1, h_2\}$$

$$R \geq \max\{h_1, h_2\}$$

$$n = n_1 + n_2 + 1 \geq 2R_1 + 1 + 2R_2 + 1 + 1 = \\ = 2(R_1 + R_2) + 2 + 1 \geq 2R + 2 + 1 = 2(R+1) + 1 \quad \square$$

ES 3

⇒ MAPPA (NO DUPLICATI, NO ORDINE) $\begin{cases} \rightarrow \text{get}(k) \\ \rightarrow \text{put}(k, v) \\ \rightarrow \text{remove}(k) \end{cases} \}^*$
 * → effettuiamo una RICERCA

(a) ⇒ $\text{put}(k, v)$:

- se $\exists$ già una entry con chiave k nella mappa allora sostituisce il suo valore con v e ritorna il vecchio valore, altrimenti inserisce la entry (k, v) nella MAPPA e ritorna null

(b) ⇒ per una mappa implementata tramite lista POSITION BASED (che si assume NON ORDINATA) ⇒ $\text{put}(k, v)$ ha complessità $O(n)$ ($n = \# \text{ENTRY della MAPPA}$), perché deve effettuare una RICERCA della CHIAVE k nella MAPPA

ES 4:

- la separazione richiede tempo $\Theta(n)$
- il mergesort richiede tempo $\Theta(n \log n)$
($\leq n \log n$, $= n \log n$ se tutti pari)
- il Quicksort richiede tempo $\Theta(n^2)$
($\leq n^2$, $= n^2$ se tutti dispari)
- merge richiede tempo $\Theta(n)$
- COMPLESSITÀ $\in \Theta(n + n \log n + n^2 + n) \in \Theta(n^2)$

SECONDA PARTE:

1. [5 punti] Sia $G(n)$ una funzione definita come segue:

- $G(0) = 1$ e $G(1) = 2$;
- $G(n-1) + 2G(n-2)$, per ogni $n > 1$.

(a) Trovare i valori di $G(n)$, per $n = 2, 3, 4, 5$.

(b) Dal punto precedente, intuire una formula chiusa per $G(n)$, per $n \geq 0$, e provarla per induzione.

2. [5 punti] Progettare un algoritmo che calcola il k -esimo elemento più piccolo di un insieme di n interi distinti, in tempo $O(n + k \log n)$ e senza ricorrere all'ordinamento.

3. [6 punti] Sia $G = (V, E)$ un grafo che rappresenta una rete sociale con n vertici ed m archi. Ogni vertice $u \in V$ ha un campo $L_V[u].influencer$ che vale 1 se u è un influencer e 0 altrimenti. Un vertice x non influencer si dice influenzabile se esiste un influencer y e un cammino tra x e y . Progettare in pseudocodice un algoritmo Influenzabili che conti il numero di vertici influenzabili in G , e analizzarne la complessità. Per avere punteggio pieno la complessità deve essere $O(n + m)$.

ES 1

• (B: $n_0 = 0, n_1 = 1 \Rightarrow n = 0 \rightarrow 1$
 $n = 1 \rightarrow 2$

• $n' > n_0 \rightarrow G(n-1) + 2 \cdot G(n-2)$

(a) $G(2) = G(1) + 2 \cdot G(0)$
 $= 2 + 2 \cdot 1 = 4$

$G(3) = G(2) + 2 \cdot G(1)$
 $= 4 + 2 \cdot 2 = 8$

$G(4) = G(3) + 2 \cdot G(2)$
 $= 8 + 2 \cdot 4 = 16$

$G(5) = G(4) + 2 \cdot G(3)$
 $= 16 + 16 = 32$

(b) $G(n) = 2^n$

→ dimostro per INDUZIONE su n ($n_0 = 1, n_1 = 1$)

• (B: $n_0 = 0 \Rightarrow G(0) = 1 = 2^n = 2^0 = 1$ OK

$n_0 + n_1 = 1 \Rightarrow G(1) = 2 = 2^{n_0 + n_1} = 2^1 = 2$ OK

• P.I: $G(n') = 2^{n'} \forall n_0 + n_1 < n' \leq n$

→ considero $G(n+1) = G(n) + 2 \cdot G(n-1)$

$= 2^n + 2 \cdot 2^{n-1} = 2^n + 2^{1+n-1}$
 $= 2^n + 2^n = 2 \cdot 2^n = 2^{n+1}$ □

ES 3

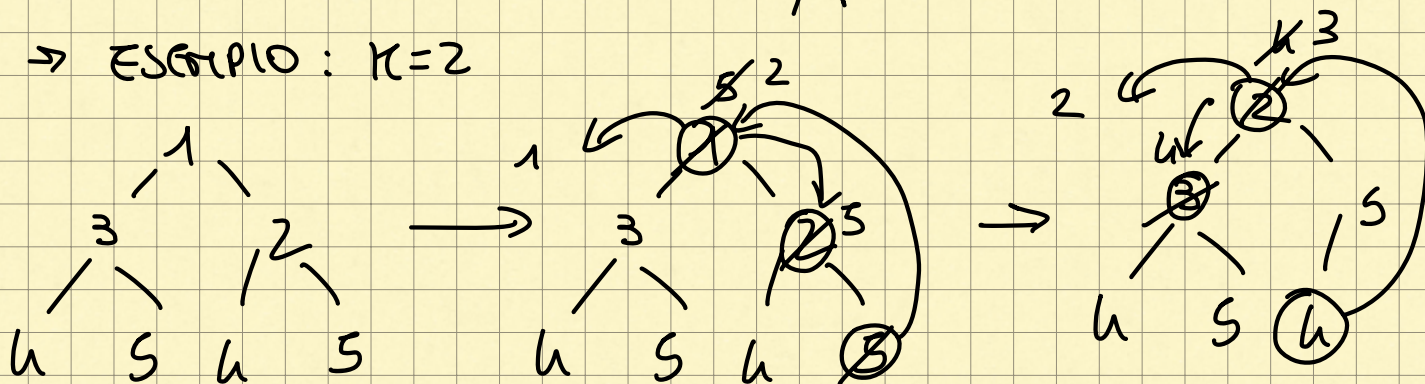
→ SPECIFICHE:

- S = sequenza di n interi distinti
- K -ESIMO ELEMENTO + PICCOLO di S
- $O(n + K \cdot \log(n))$
- NO ORDINAMENTO

→ IDEA:

- inserisco gli n interi distinti in una SEQUENZA S (tempo $O(n)$)
- uso un MIN HEAP da S con APPROCCIO BOTTOM-UP $\in O(n)$ ($K < \min\{K_1, K_2\}$)
- uso strategia simile a remove K in() di una PQ usando la DHB K VOLTE per prendere i K ELEMENTI + PICCOLI, ruotando solo l'ultimo

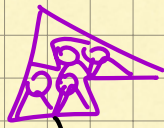
→ ESEMPIO: $K=2$



- A: K -Min Element
- I: insieme di n interi distinti
- O: K -esimo intero + piccolo

Inserisci gli n interi in un array $P[0], \dots, P[n-1]$

// BOTTOM-UP



$S[j:n]$ FORESTIAN
HEAP
 $j \leftarrow \lfloor n/2 \rfloor$ down to 0
($i > 1$)

for $j \leftarrow \lfloor n/2 \rfloor$ down to 1 do {

$i \leftarrow j$;

$k \leftarrow \text{index_min}(\text{Root}(P, j))$

while $(z_i \leq \text{last})$ AND $P[i].gk() > P[k].gk())$ do {

...

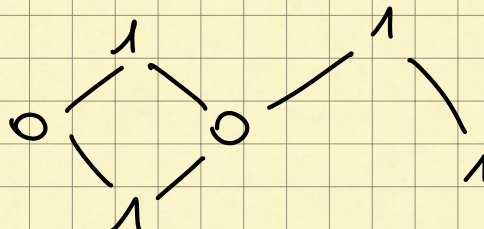
ESU

3. [6 punti] Sia $G = (V, E)$ un grafo che rappresenta una rete sociale con n vertici ed m archi. Ogni vertice $u \in V$ ha un campo $L_V[u].\text{influencer}$ che vale 1 se u è un influencer e 0 altrimenti. Un vertice x non influencer si dice influenzabile se esiste un influencer y e un cammino tra x e y . Progettare in pseudocodice un algoritmo Influenzabili che conti il numero di vertici influenzabili in G , e analizzarne la complessità. Per avere punteggio pieno la complessità deve essere $O(n + m)$.

→ SPECIFICHE:

- $G = (V, E)$ n VERTICI, m ARCHI
- $\forall u \in V$ $u.\text{influencer} = \begin{cases} 1 & \text{se INF} \\ 0 & \text{se NON INF} \end{cases}$
- se u ha $u.\text{influencer} = 0$ è INFLUENZABILE da v se $\exists$ un cammino tra u e v
- conta # vertici influenzabili
- COMPLESSITÀ $O(n + m)$

→ ESEMPIO:



→ IDEA: uso BFS(G, s) ES(ESA ($\in \Theta(n+m)$))

con modifica per tenere traccia dei CAMMINI

- ogni nodo ha un campo $v.parent$ e $v.edge$
- quando v scopre u tramite $e = (u, v) \Rightarrow$
 $u.parent \leftarrow v, u.edge \leftarrow e$

?

• NON SO BENE COME FUNZIONA

ESEMPIO SCRITTO 3

PRIMA PARTE

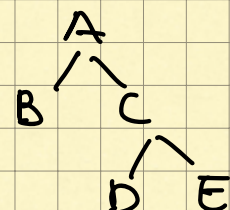
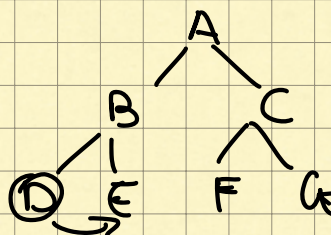
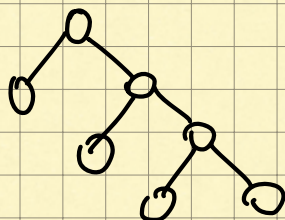
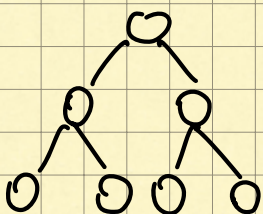
1. [3 punti] Definire come problema computazionale il problema di trovare, dato un intero $n > 1$, un intero k con $1 < k < n$ tale che n è divisibile per k , se un tale k esiste.
2. [3 punti] Sia T un albero binario proprio con $n > 1$ nodi e radice r . Sia u il figlio sinistro di r . Qual è il nodo successore di u nella visita in postorder di T .
3. [3 punti] Si consideri una coda con priorità P , implementata tramite *heap*, inizialmente vuota. Si supponga di eseguire le seguenti operazioni nell'ordine dato: $P.insert(5, \cdot)$, $P.insert(3, \cdot)$, $P.insert(6, \cdot)$, $P.insert(4, \cdot)$, $P.insert(7, \cdot)$, $P.insert(2, \cdot)$, $P.removeMin()$. Disegnare lo heap risultante (come albero, riportando solo le chiavi) alla fine della sequenza di operazioni.
4. [4 punti] Dimostrare che un Multi-Way Search Tree contenente n entry ha $n + 1$ foglie.
5. [3 punti] È vero che applicando l'algoritmo probabilistico Randomized QuickSort a una sequenza S di n chiavi distinte ordinata in senso decrescente, vengono eseguite $\Theta(n^2)$ operazioni? Motivare la risposta.

ES 1 :

- $\pi = \{(i, s) : i \in I, s \in S\}$
- $I = \{i \in \mathbb{Z} : i > 1\}$
- $S = \{s \in \mathbb{Z}, 1 < s < n : \frac{i}{s} \in \mathbb{Z}\}$

ES 2 :

- $T = ABP \leftrightarrow$ ogni nodo interno ha 2 figli



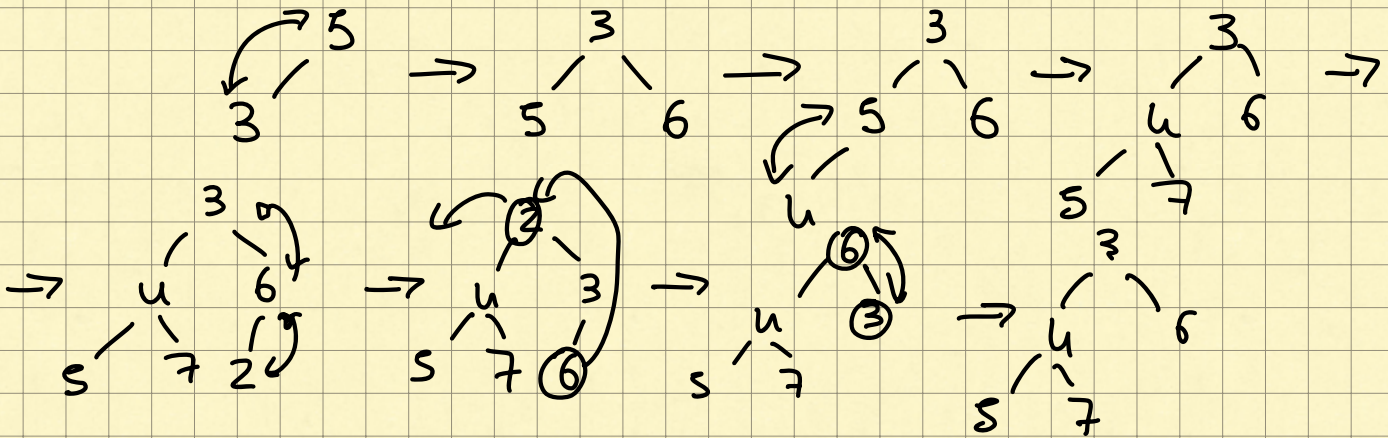
$D \rightarrow G \rightarrow B \rightarrow A \rightarrow \dots$ $B \rightarrow A \rightarrow \dots$

- postorder $(\tau, v) \in \Theta(n) \Leftrightarrow$ prima visita ricorsiva nei sott Alberi radicati nei figli, poi il padre
- $\Rightarrow$ il nodo successore di u nella visita in postorder è la radice r .

ES 3:

3. [3 punti] Si consideri una coda con priorità P , implementata tramite heap, inizialmente vuota. Si supponga di eseguire le seguenti operazioni nell'ordine dato: $P.insert(5, \cdot)$, $P.insert(3, \cdot)$, $P.insert(6, \cdot)$, $P.insert(4, \cdot)$, $P.insert(7, \cdot)$, $P.insert(2, \cdot)$, $P.removeMin()$. Disegnare lo heap risultante (come albero, riportando solo le chiavi) alla fine della sequenza di operazioni.

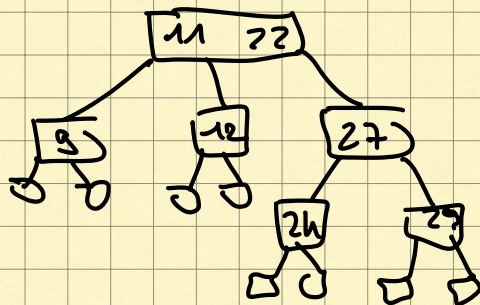
$\Rightarrow$ MIN HEAP $\Rightarrow$ HOP: $\pi_1 \leq \min\{\pi_1, \pi_2\}$



ES 4

$\Rightarrow$ MWST n entry ha $m = n+1$ foglie

$\Rightarrow$ MWST



($\neq$ - NODE)

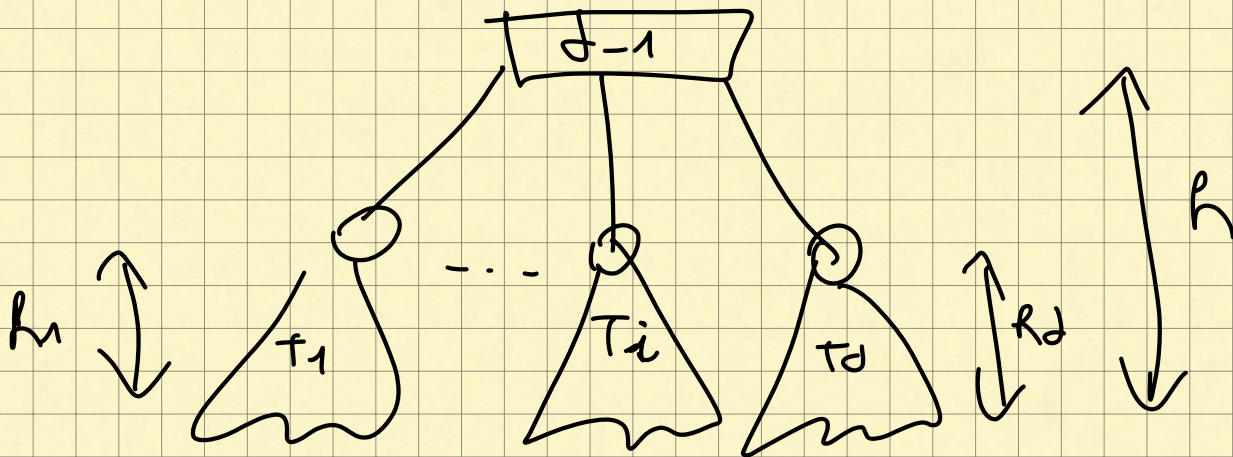
$$\pi_{i-1} < e.getkey() < \pi_i$$

⇒ INDUZIONE sull'ALTEZZA R ($R_0=0, R=0$)

• CB: $R=R_0=0 \rightarrow \square \quad 1=0+1$
 $1=1 \quad 0$

• PI: proprietà vera per R' con $R_0 < R' \leq R$

⇒ considero MUST di altezza $R+1$



$$R = \{R_1, \dots, R_d\} \quad (\text{da def di altezza})$$

$$n = \sum_{i=1}^d h_i + d - 1$$

$$m = \sum_{i=1}^d m_i, \quad \textcircled{HI} \quad m_i = n_i + 1$$

$$m = \sum_{i=1}^d n_i + d \quad (-1 + 1)$$

$$m = n'' + 1 \quad \square$$

ES 5

⇒ no, vengono eseguite in media $\Theta(n \log n)$ operazioni (vedi complessità Randomized AS.)

⇒ in realtà potrebbe essere anche di sì, perché

la seq è decrescente, però in questo momento non ho voglia di perderci tempo.

SECONDA PARTE

1. [8 punti] Dato un albero binario proprio T , definiamo il level numbering dei nodi di T come segue:

- se v è radice di T , allora l'indice $\ell(v)$ di v nel level numbering è 1, cioè $\ell(v) = 1$;
- se v non è radice di T , sia u il padre di v : allora $\ell(v) = 2\ell(u)$ se v è figlio sinistro di u ; $\ell(v) = 2\ell(u) + 1$ se v è figlio destro di u .

Si vuole progettare un algoritmo ricorsivo che salvi in un campo $v.\text{indexLN}$ l'indice $\ell(v)$ di v nel level numbering dei nodi di T , per ogni nodo $v \in T$.

- (a) Si fornisca lo pseudocodice per l'algoritmo richiesto.
- (b) Si analizzi la complessità dell'algoritmo.

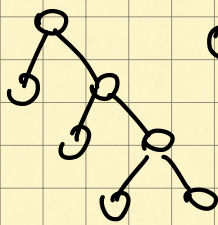
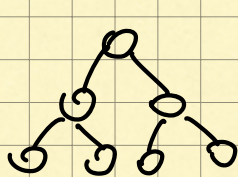
Per ricevere il massimo dei punti l'algoritmo deve avere complessità in tempo $O(n)$, dove n è il numero di nodi di T .

2. [8 punti] Sia $G = (V, E)$ un grafo con n vertici ed m archi.

- (a) Progettare un algoritmo che dato G ed un intero $k > 0$, per ogni nodo $v \in V$ salva in una variabile $v.\text{numNeighbors}$ il numero di nodi a distanza $\leq k$ da v .
- (b) Analizzare la complessità dell'algoritmo.

ES 1

• ABP



$$\textcircled{1} m = n - 1$$

$$\textcircled{2} R + 1 \leq m \leq 2^R$$

$$\textcircled{3} \log(n+1) - 1 \leq R \leq \frac{(n-1)n}{2}$$

• LEVEL NUMBERING:

$$\bullet v = \text{radice} \rightarrow \ell(v) = 1$$

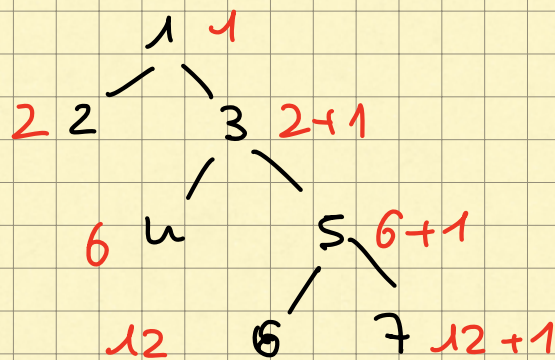
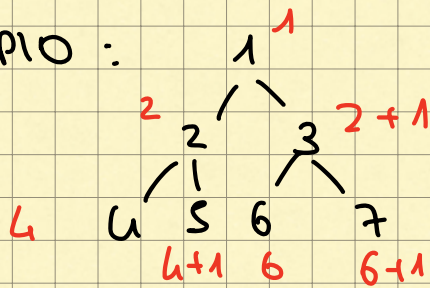
• $v \neq \text{radice}$:

$$\bullet \text{ se FIGLIO SX} \rightarrow \ell(v) = 2 \cdot \ell(T.\text{parent}(v))$$

• se $F(G) \neq \emptyset \Rightarrow e(v) = 2 - e(u) + 1$

- $\forall$ nodo $v \in T$ scrivi in v . index ($N \leftarrow e(v)$)
- COMPLESSITÀ $\Theta(n)$

• ESEMPIO:



→ IDEA:

→ crea un ALGORITMO che si basa su:

$$\text{index}(T, v) \in \Theta\left(\sum_{u \in T_v} (t_u + u + 1)\right) \in \Theta(n)$$

...

ES 2

- $G = (V, E)$ grafo
- $K > 0$ intero
- $\forall v \in V$ salva in v . numNeighbors il # NODI a DISTANZA $\leq K$ da v

...

→ IDEA:

→ uso BFS(G, s) GENERALIZZATA $\in \Theta(n+m)$,
SPECIALIZZATA per trovare i CAMMINI MINIMI
di OGNI NODO da s , ovvero $\forall$ nodo uso i campi
 u . parent e u . edge.

→ quando v SCOPRE u tramite $e = (u, v) \Rightarrow$
 u . parent $\leftarrow u$, u . edge $\leftarrow e = (u, v)$

- alla fine dell'esecuzione della BFS sul nodo v , trovo i vicini con distanza $\leq H$.
- poi ripeto il procedimento $\forall$ nodo $v \in V$.
- complessità $\Theta(n(n+m))$

?

ESEMPIO SCRITTO 4

PRIMA PARTE

1. [3 punti] Si consideri un ciclo

for $i \leftarrow 1$ **to** n **do** {*corpo del ciclo*}.

Dato un invariante per il ciclo dire brevemente cosa bisogna fare per provare che alla fine del ciclo vale una certa proprietà S_{end} .

2. [4 punti] Dimostrare che la complessità di n inserimenti in una Priority Queue implementata tramite uno Heap e inizialmente vuota è $\Theta(n \log n)$.
3. [3 punti] Una funzione hash è caratterizzata da due componenti: un hash code e una compression function. Descrivere brevemente il ruolo di ciascun componente e indicare un buon hash code per stringhe di caratteri.
4. [3 punti] Sia S una sequenza di n chiavi intere da ordinare. Dire quale tra gli algoritmi di ordinamento che conoscete usereste nei seguenti casi, specificando la complessità risultante: (i) le chiavi da ordinare appartengono all'intervallo $[1, n^2]$; (ii) esistono 3 chiavi, tolte le quali S risulta già ordinata. Motivare le risposte.
5. [3 punti] Sia $G = (V, E)$ un grafo non diretto con n vertici ed m archi.
- (a) Se G è connesso, qual è il valore minimo che m può assumere?
- (b) Dimostrare l'affermazione precedente.

ES1

- si dimostra che l'invariante è vero all'inizio del ciclo
- si dimostra che se è vero alla fine dell'iteratione $i-1 \Rightarrow$ è vero anche alla fine dell'iteratione i
- si dimostra che se è vero alla fine dell'iteratione $n-1$ (ultima iteration) $\Rightarrow$ è vero anche alla fine del ciclo (stato finale S_{end})

ES 2

→ insert (k, v) di PQ con HEAP ha complessità
 $\Theta(\log j)$ perché: ($j = \#$ nodi in PQ)



- $R_0 \leq R$ iterazioni del ciclo while della UP-HEAP BUBBLING, di costo $\Theta(1) \Rightarrow \Theta(R)$
- $R_0 = R //$
di costo $\Theta(1)$, se $k < k_{\min}$
 $\Rightarrow \Omega(R)$

$\Rightarrow \Theta(R) \in \Theta(\log_2(j))$ perché $HEAP = ABC \Rightarrow$
 $R = \lfloor \log_2(j) \rfloor$

→ se eseguo n insert la complessità
 $\Theta\left(\sum_{j=1}^n \log_2(j)\right) \in \Theta(n \log(n))$

(ANALOGO a APPROCCIO TOP DOWN)

ES 3

NON LO SO BRO, SPIAZZE

ES 4

- (i) piani nell'intervallo $[0, n^2] \rightarrow \text{MergeSort} \in \Theta(n \log n)$, perché $\text{BUCKET SORT} \in \Theta(h+N)$
con $N = n^2 \Rightarrow \in \Theta(n^2)$
- (ii) seq quasi ordinata $\Rightarrow \text{insertion sort} \in \Theta(n^2)$
Po $K = O(\log_2 n)$ inversioni

ES 5

- (a) $G \text{ CONNESSO} \Rightarrow m \geq n-1$
- (b) • se $G \text{ CONNESSO}$ e SENZA CICLI $\Rightarrow \text{ALBERO} \Rightarrow m = n-1$
• se $G \text{ CONNESSO}$ e con CICLI $\rightarrow$ segui il while:
while $(\exists \text{ un CICLO } C \text{ in } G)$ do {
 rimuovi un arco di C in G
}
- $\rightarrow$ alla fine di ogni iterazione G è vero che
 G è connesso
- $\rightarrow$ alla fine se il while G è connesso e
senza cicli (ALBERO) $\Rightarrow m = n-1$
- $\Rightarrow$ originariamente $m \geq n-1$

SECONDA PARTE

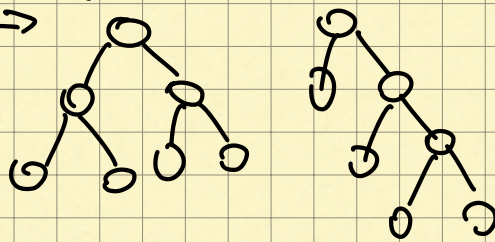
- [8 punti] Si vuole progettare un algoritmo *ricorsivo* deepLeaf per trovare la foglia più profonda in un albero binario proprio T .
 - Descrivere tramite pseudocodice la generica invocazione di deepLeaf, specificandone con attenzione l'input e l'output.
 - Analizzare la complessità di deepLeaf.
- [8 punti] Sia T un $(2,4)$ -Tree contenente n entry con chiavi intere distinte.
 - Progettare un algoritmo iterativo efficiente che determina la più grande chiave negativa in T . Se in T non esistono chiavi negative, l'algoritmo deve restituire "no negative key".
 - Analizzare la complessità dell'algoritmo del punto precedente

ES 1

→ SPECIFICHE: (RICORSIVO)

- foglia + profonda in T

• T ABP $\Leftrightarrow$



ogni nodo interno ha esattamente 2 figli

- $\text{inorder}(T, v), \text{preorder}(T, v), \text{postorder}(T, v) \in$

$$\Theta\left(\sum_{u \in Tv} (t_u + c_u + 1)\right) \in \Theta(n)$$

- $\text{depth}(T, v) \in \Theta(d_v + 1)$

- $\text{height}(T, v) \in \Theta(n)$

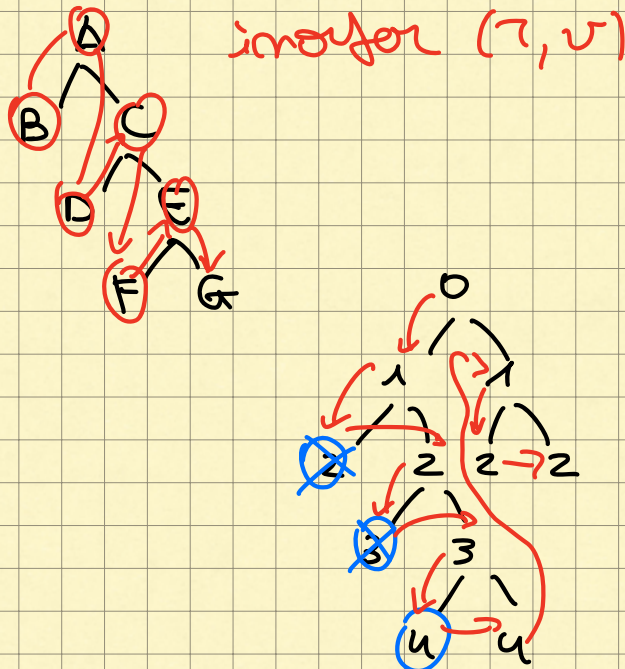
- $\text{height}(T) = \max_{u \text{ foglia}} \{ \text{depth}(T, u) \}$

① $m = n - m + 1$

$$\textcircled{2} \quad h+1 \leq m \leq 2^h$$

$$\textcircled{3} \quad \log_2(n+1)-1 \leq h \leq \frac{n(n-1)}{2}$$

→ ESEMPIO



→ IDEA:

- $\text{depth}(T, v) = 1 + \text{depth}(T, \text{T.parent}(v))$
- faccio visita in PRE-ORDER di T e man mano che scendo tengo TRACCA delle PROFONDITÀ
- Algoritmo: $\text{deepLeaf}(T, v, u)$ 1^{a} INV $v = r, (u = \text{null})$
- Input: nodo $v \in$ albero binario proprio T , ogni $u \in T$ ha un campo $u.\text{depth}$ che deve essere impostato per tutti gli antenati di u e che vale 0 se non impostato, profondità di u in T altrimenti, $u = \text{logica} + \text{profondità in } T$
- Output: foglia + profondità di Tv in T

// PREORDER

```
if (T.isRoot(v)) then v.depth  $\leftarrow$  0;  
else v.depth  $\leftarrow$  1 + T.parent(v).depth();  
if (v.children.isEmpty()) {  
    if ((a = null) OR (v.depth < a.depth())) then a  $\leftarrow$  v;  
    for each (w  $\in$  T.children(v)) do {  
        keepLeaf(T, v, a);  
    }  
}
```

(b) COMPLESSITA' $\in \Theta(n)$ da PREORDER

2. [8 punti] Sia T un $(2,4)$ -Tree contenente n entry con chiavi intere distinte.

- Progettare un algoritmo iterativo efficiente che determina la più grande chiave negativa in T . Se in T non esistono chiavi negative, l'algoritmo deve restituire "no negative key".
- Analizzare la complessità dell'algoritmo del punto precedente

(a)

→ SPECIFICHE:

- $(2-4)$ TREE
- n - ENTRY con CHIAVI DISTINTE
- ALGORITMO ITERATIVO
- + GRANDE CHIAVE NEGATIVA in T , se $\exists$
- "no neg key" altrimenti

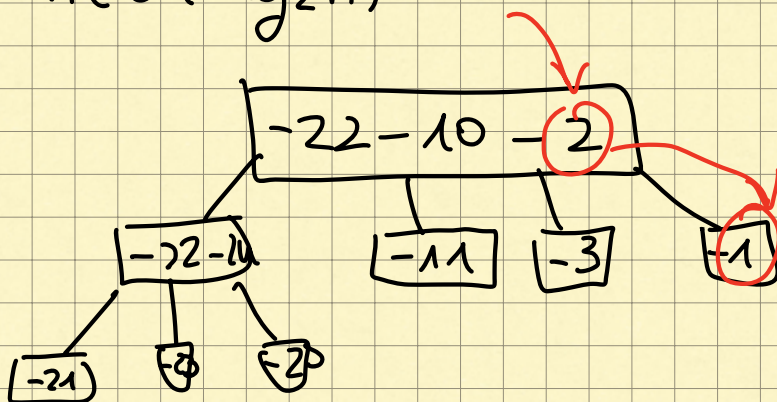
→ $(2-4)$ TREE $\leftrightarrow$ MWST (con $2 \leq d \leq 4$) $\leftrightarrow m = n - 1$

$\Downarrow$

MWTS $(n, v) \in \Theta(d_{\max} \cdot R)$

$R \in \Theta(\log_2 n)$

$\in \Theta(\log_2(n))$



scendo a dx

→ IDEA: prendo spunto dal MWTS

... ?


```

if (k is External()) then return "no neg key"
i ← -1;
i ← indice entry di v < 0, con chiave maggiore tra le due d-1
if (i ← -1) then {
  if k = null return "no neg key";
  return k;
}
k = v.i.getKey();
return MIXTS(k, v.i)

```

↑
 dovremmo farlo iterativo, in
 ogni caso si può convertire
 in iterativo

→ oppure "scendi sempre a dx dell'albero finché trovi chiavi < 0"

ESEMPIO SCRITTO 5

PRIMA PARTE

1. [3 punti] Si definisca come *problema computazionale* Π il seguente problema: dato un array di interi determinare il numero di celle con valore maggiore o uguale ad un dato intero k .
2. [4 punti] Si consideri la visita in *preorder* di un albero T di n nodi eseguita invocando $\text{preorder}(T, T.\text{root}())$. Per ogni nodo $u \in T$, sia c_u il numero di figli di u , e sia t_u il costo della visita di u . Si ricordi che $\sum_{u \in T} c_u = n - 1$.
 - (a) Dire quanti nodi ha l'albero della ricorsione associato a $\text{preorder}(T, T.\text{root}())$, e qual è il costo di ciascun nodo.
 - (b) Sia $t_u \in \Theta(\log n)$ per ogni $u \in T$. In base al punto precedente, determinare la complessità di $\text{preorder}(T, T.\text{root}())$.
3. [4 punti] Si consideri l'array di entry: $P = [(10, *), (8, *), (6, *), (7, *), (5, *), (2, *)]$, e l'approccio *top-down* per la costruzione di uno *heap* a partire da P . Mostrare l'albero associato all'array alla fine di ogni iterazione dell'approccio *top-down*. (È sufficiente mostrare le chiavi delle entry.)
4. [5 punti] Si consideri un grafo $G = (V, E)$. Dimostrare che $\text{DFS}(G, s)$ visita tutti i nodi della componente connessa di s .

ES 6:

① Poiché $\text{DFS}(G, s)$ visita tutti i nodi della componente connessa s . (analogo a BFS)

② $\Rightarrow$ per ASSURDO suppongo che $\exists$ un nodo $v \in C_s$ NON VISITATO.

③ $\Rightarrow$ siccome $v \in C_s \Rightarrow \exists$ un CAMMINO P :

$s = u_0, \dots, u_k = v$ con $u_0, \dots, u_k \in C_s$ e $\exists u_i \neq u_0, = 1^\circ$ nodo NON VISITATO ($u_i.\text{id} = 0$)

② $\rightarrow$ ma u_{i-1} è VISITATO $\rightarrow u_{i-1}$ inserito in
 lista $L_i \rightarrow$ quando viene analizzata
 tale lista $\rightarrow$ analizzati gli archi
 che indicano su $u_{i-1} \rightarrow$ viene preso
 $e = (u_{i-1}, u_i)$ che fa $e.label = null$ e
 $u_i.id = 0 \rightarrow e.label \leftarrow 0 \in$ e $u_i.id \leftarrow 1$,
 che COMINCIA l'IPOTESI ASSUNTA

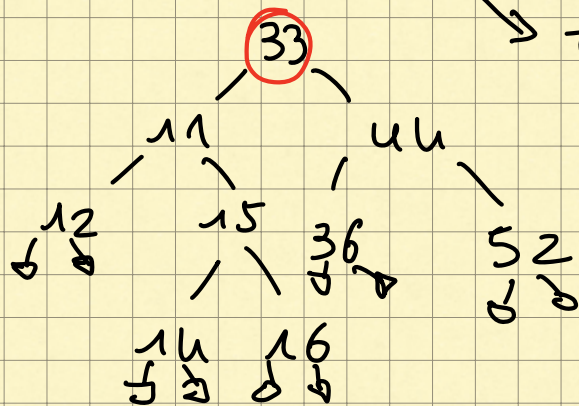
SECONDA PARTE

1. [8 punti] Sia T un albero binario di ricerca che implementa una mappa le cui entry sono coppie (t, i_t) , dove t (la chiave) rappresenta un istante di tempo in cui è avvenuto un terremoto, e i_t (il valore) rappresenta l'intensità del terremoto. Per ogni nodo $v \in T$ esiste un campo $v.max$ che riporta la massima intensità dei terremoti rappresentati dalle entry di T_v (sottoalbero con radice v).
 - (a) Progettare un algoritmo ricorsivo CheckGG che, dati un tempo t e un valore di intensità i , restituisce yes se c'è stato un terremoto di intensità $> i$ in un istante di tempo $\geq t$, altrimenti restituisce no. Specificare chiaramente l'input e l'output dell'algoritmo.
 - (b) Analizzare la complessità dell'algoritmo del punto precedente.
2. [8 punti] Siano $S_1[0 \div n - 1]$ and $S_2[0 \div n - 1]$ due array ordinati di n chiavi intere ciascuno. In ogni array le chiavi sono distinte, ma una chiave può comparire in entrambi gli array.
 - (a) Progettare un algoritmo CountNeg, **contenente un solo ciclo**, che determina il numero di chiavi negative distinte che compaiono in S_1 e/o S_2 . Ad esempio, se $S_1 = [-3, -2, 0, 34]$ e $S_2 = [-7, -2, 34, 45]$ l'algoritmo restituisce 3 (chiavi negative distinte: -7, -3, -2). Per avere punteggio pieno, l'algoritmo deve utilizzare spazio costante, oltre a S_1 e S_2 .
 - (b) Specificare un opportuno invariante che serva per provare la correttezza dell'algoritmo.

ES 1

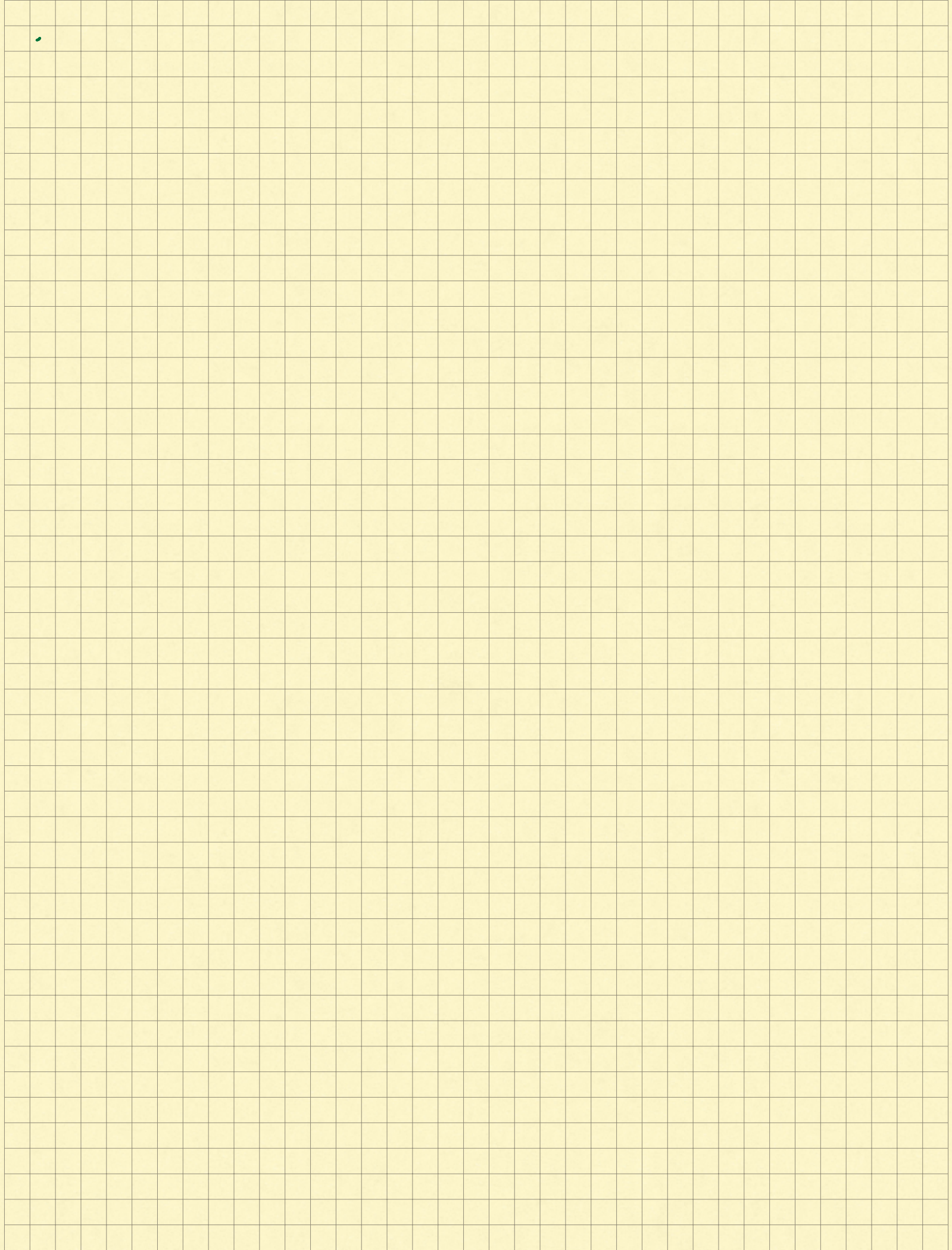
$$\nearrow ABP \Rightarrow h \in \Theta(n)$$

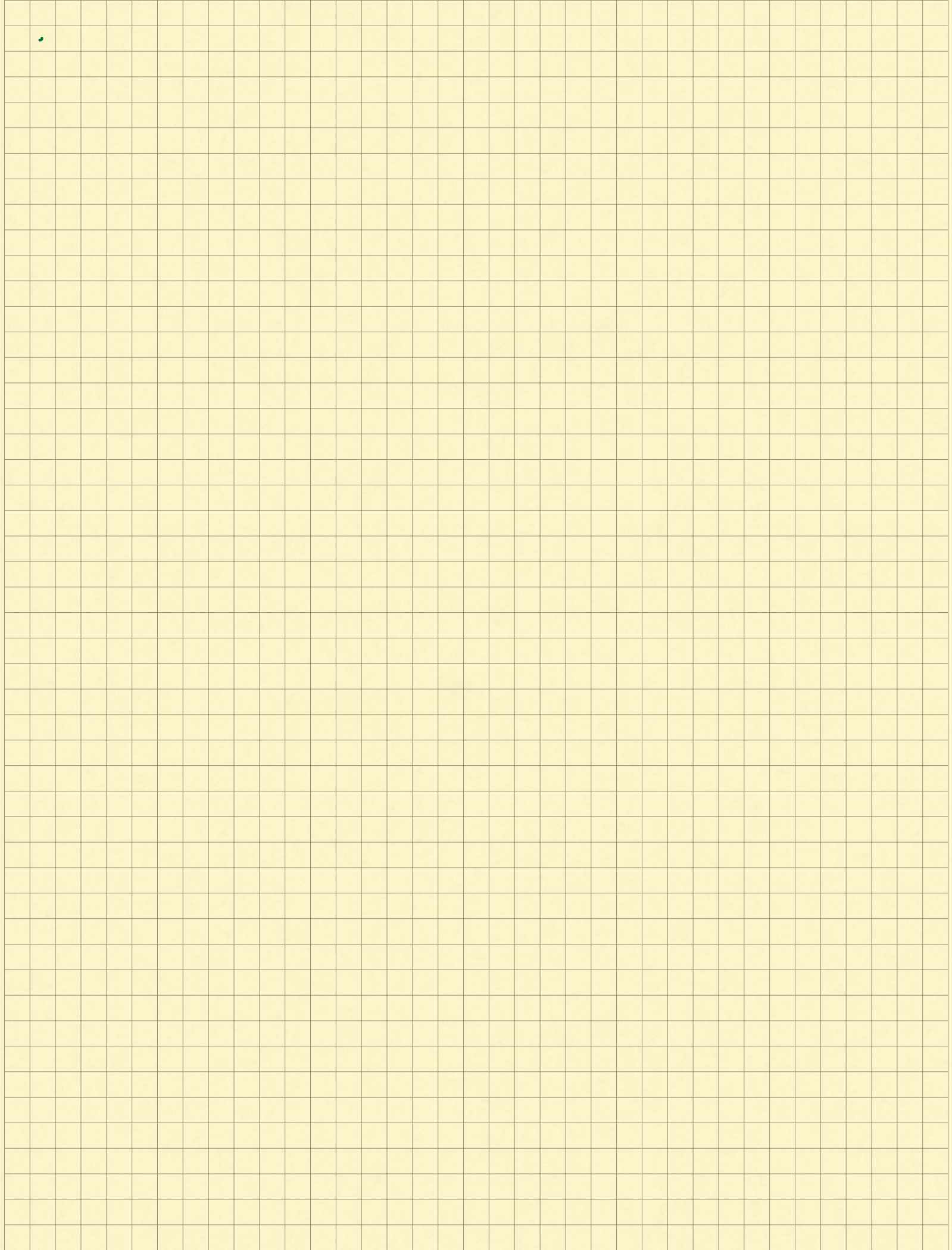
- $T = ABR \Leftrightarrow ST \Leftrightarrow$ (lavori Tv se $x < v$, altrimenti $v+x > v$)
 $\searrow TS \in \Theta(R) \in \Theta(n)$



- $\forall v \in T \quad v.MAX = \text{max intensita' modi sostenibili}$
 Tv
- RECURSIVO CHECK GG
- $(i, t) \rightarrow se \quad 1$

$\Rightarrow$ IDEA: TreeSearch (H, v) MODIFICATO
 di (H) .





ESERCIZIO SCRITTO 6

1. [5 punti] Siano $[a_0, b_0], [a_1, b_1], \dots, [a_{n-1}, b_{n-1}]$ n intervalli sulla retta reale, con $a_i < b_i$. Gli intervalli sono ordinati in base all'estremo sinistro ($a_0 \leq a_1 \leq \dots \leq a_{n-1}$) ma possono sovrapporsi. Il seguente ciclo determina la lunghezza massima di un segmento continuo della retta reale coperto interamente dagli intervalli.

```
Acurr ←  $a_0$ ; Bcurr ←  $b_0$ ; max_length ← ( $b_0 - a_0$ );  
for  $i \leftarrow 1$  to  $n - 1$  do  
    if ( $a_i > Bcurr$ ) then  
        Acurr ←  $a_i$ ;  
        Bcurr ←  $b_i$ ;  
    else Bcurr ←  $\max\{Bcurr, b_i\}$ ;  
    max_length ←  $\max\{\max\_length, Bcurr - Acurr\}$ ;  
return max_length
```

Trovare un invariante per il ciclo for, che specifichi che cosa rappresentano le variabili Acurr, Bcurr, max_length alla fine di ciascuna iterazione.

2. [6 punti] Sia T un albero binario proprio dove ogni nodo $v \in T$ contiene un valore binario. Un nodo $v \in T$ si dice *3-balanced*, se $|n_0(v) - n_1(v)| \leq 3$, dove $n_0(v)$ e $n_1(v)$ denotano il numero di 0 ($n_0(v)$) e 1 ($n_1(v)$) nel sottoalbero T_v . Si progetti in pseudocodice un algoritmo ricorsivo **All3Balanced** per determinare se *tutti i nodi di T sono 3-balanced*, con complessità lineare nel numero di nodi di T .
3. [5 punti] Sia $G = (V, E)$ un grafo non diretto, non pesato e connesso, con $n = |V|$ e $m = |E|$, dove ogni vertice v ha un bit $v.C$ che vale 1 se v è un *centro*, e 0 altrimenti. Sia $C \subset V$ l'insieme dei centri e $k = |C|$. Progettare in pseudocodice un algoritmo che dato un tale grafo G calcoli, per ogni vertice $v \in V$, la distanza dal centro più vicino a lui, ovvero $\min_{s \in C} d(v, s)$, dove $d(v, s)$ è il numero di archi nel cammino minimo da v a s , memorizzandola in un campo $v.distC$. Analizzare la complessità dell'algoritmo (per avere punteggio massimo deve essere $O(km)$).